



Pandora Solana Security Audit Report

April 17, 2026



Contents

1 Introduction

[1.1 About Pandora](#)

[1.2 Source Code](#)

[1.3 Revision History](#)

2 Overall Assessment

3 Vulnerability Summary

[3.1 Overview](#)

[3.2 Security Level Reference](#)

[3.3 Vulnerability Details](#)

4 Appendix

[4.1 About AstraSec](#)

[4.2 Disclaimer](#)

[4.3 Contact](#)

1 Introduction

1.1 About Pandora

Pandora Protocol is a decentralized prediction market protocol enabling users to create and trade binary outcome markets. The system supports both constant product AMM and pari-mutuel market mechanisms, with automated pricing, liquidity provision, and oracle-based resolution for efficient market operations.



1.2 Source Code

The following source code was reviewed during the audit:

▶ <https://github.com/RealWagmi/pandora-solana-contracts.git>

▶ Commit: 8094875

And this is the final version representing all fixes implemented for the issues identified in the audit:

▶ <https://github.com/RealWagmi/pandora-solana-contracts.git>

▶ Commit: 3697ae5

We assume that the price oracle used in the current implementation is robust and reliable, and that asset prices are provided in a timely manner. The implementation of the price oracle itself is not included in the scope of this audit.

1.3 Revision History

Version	Date	Description
v1.0	April 17, 2026	Initial Audit

2 Overall Assessment

This report has been compiled to identify issues and vulnerabilities within the Pandora protocol. Throughout this audit, we identified a total of 6 issues spanning various severity levels. By employing auxiliary tool techniques to supplement our thorough manual code review, we have discovered the following findings.

Severity	Count	Acknowledged	Won't Do	Addressed
Critical	1	—	—	1
High	—	—	—	—
Medium	2	1	—	1
Low	2	—	—	2
Informational	1	—	—	1
Undetermined	—	—	—	—

3 Vulnerability Summary

3.1 Overview

Click on an issue to jump to it, or scroll down to see them all.

G-1

[Unchecked U128-To-U64 Cast in Exact-Out Swap](#)

M-1

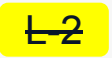
[Potential Reserve-Vault Mismatch on Sell](#)

M-2

[Potential Risks Associated with Centralization](#)

L-1

[Revisited Chainlink Controller in set_price\(\)](#)

L-2

[Mandatory Opposite-Side ATA Blocks Single-Sided Sellers](#)

H-1

[Revisited Privileged Account Typing](#)

3.2 Security Level Reference

In web3 smart contract audits, vulnerabilities are typically classified into different severity levels based on the potential impact they can have on the security and functionality of the contract. Here are the definitions for critical-severity, high-severity, medium-severity, and low-severity vulnerabilities:

Severity	Acknowledged
C-X (Critical)	A severe security flaw with immediate and significant negative consequences. It poses high risks, such as unauthorized access, financial losses, or complete disruption of functionality. Requires immediate attention and remediation.
H-X (High)	Significant security issues that can lead to substantial risks. Although not as severe as critical vulnerabilities, they can still result in unauthorized access, manipulation of contract state, or financial losses. Prompt remediation is necessary.
M-X (Medium)	Moderately impactful security weaknesses that require attention and remediation. They may lead to limited unauthorized access, minor financial losses, or potential disruptions to functionality.
L-X (Low)	Minor security issues with limited impact. While they may not pose significant risks, it is still recommended to address them to maintain a robust and secure smart contract.
I-X (Informational)	Warnings and things to keep in mind when operating the protocol. No immediate action required.
U-X (Undetermined)	Identified security flaw requiring further investigation. Severity and impact need to be determined. Additional assessment and analysis are necessary.

3.3 Vulnerability Details

3.3.1 [C-1] Unchecked U128-To-U64 Cast in Exact-Out Swap

Target	Category	IMPACT	LIKELIHOOD	STATUS
am_math.rs	Business Logic	High	High	Addressed

`get_amount_in()` computes the required input in u128 then coerces to u64 with `as u64` without proving the value fits `u64::MAX`. Values above `u64::MAX` truncate to the low 64 bits. Downstream, `swap_exact_out()` uses this as the actual transfer amount while still delivering the user's `amount_out`, so an attacker can drastically underpay input relative to output. `calculate_swap_exact_out_internal()` repeats the unsafe pattern for `amount_in_without_fee`, corrupting fee decomposition. `get_amount_out_with_fee_numer()` should use explicit safe conversion for consistency.

```
pandora-solana-contracts-main - am_math.rs

521
522 pub fn get_amount_in(
523     market: &AmmMarket,
524     amount_out: u64,
525     reserve_in: u64,
526     reserve_out: u64,
527 ) -> AnchorResult<u64> {
528     ...
529
530     let amount_in = (numerator as u128)
531         .checked_add(denominator - 1).ok_or(ErrorCode::MathOverflow)?
532         .checked_div(denominator).ok_or(ErrorCode::MathOverflow)? as u64;
533
534     Ok(amount_in)
535 }
```

Remediation Use `u64::try_from()` on the computed u128 value (e.g. `u64::try_from(amount_u128).map_err(|_| ErrorCode::MathOverflow)?`) or an explicit `> u64::MAX` check after division.

3.3.2 [M-1] Potential Reserve-Vault Mismatch on Sell

Target	Category	IMPACT	LIKELIHOOD	STATUS
sell.rs	Business Logic	Medium	Medium	Addressed

In `sell()`, reserves are updated using the full `swapped_amount` before burn amounts are finalized, but when `token_remaining < swapped_amount`, the protocol can only burn `pairs_to_burn = token_remaining` on both outcome vaults. This means the reserve fields (`yes_reserves` / `no_reserves`) reflect a larger output-side reduction than the actual counter-vault burn, creating a reserve-vault mismatch (`swapped_amount - pairs_to_burn`) that can mislead downstream logic if reserves are assumed to track vault balances 1:1.

```
pandora-solana-contracts-main - sell.rs
94 pub fn apply(ctx: &mut Context<Sell>, params: &SellParams) -> Result<()> {
95     ...
96     let amount_to_swap = calculate_optimal_sell_swap(params.token_amount_in, params.is_yes, reserve_yes, reserve_no, fee_number)?;
97
98     let swapped_amount = get_amount_out_with_fee_number(
99         amount_to_swap,
100         if is_yes { reserve_yes } else { reserve_no },
101         if is_yes { reserve_no } else { reserve_yes },
102         fee_number,
103     );
104
105     update_reserves_after_swap(market, is_yes, amount_to_swap, swapped_amount, reserve_yes, reserve_no)?;
106
107     let token_remaining = sub_checked(params.token_amount_in, amount_to_swap)?;
108     let pairs_to_burn = if token_remaining < swapped_amount {
109         token_remaining
110     } else {
111         swapped_amount
112     };
113     ...
114     token::burn(/* yes_vault */, pairs_to_burn)?;
115     token::burn(/* no_vault */, pairs_to_burn)?;
116     ...
117 }
```

Remediation When `token_remaining < swapped_amount`, consider transferring the counter-asset shortfall `dust = swapped_amount - pairs_to_burn` from the relevant outcome vault to the seller's ATA.

3.3.3 [M-2] Potential Risks Associated with Centralization

Target	Category	IMPACT	LIKELIHOOD	STATUS
Multiple Contracts	Security	High	Low	Acknowledged

The Pandora protocol relies on multiple privileged accounts that possess extensive control over critical operations, introducing notable centralization risks. These accounts, assigned distinct roles, can unilaterally influence the protocol's functionality and integrity. Below are key examples of privileged functions and their associated roles:

- ➔ **Program Upgrade Authority:** Controls the deployment and upgrade process of the entire smart contract system, including the ability to modify protocol logic, update IDL definitions, and change core functionality.
- ➔ **Owner (config.owner):** Governs protocol administration, including owner/treasury changes, fee and pause settings, and authorization list management (operators, collaterals, feed IDs).
- ➔ **Operator (operator_auth):** Submits poll answers during the normal resolution window (set_answer), directly influencing market outcomes before finalization.

```
pandora-solana-contracts-main - lib.rs
37 pub fn set_treasury(mut ctx: Context<SetTreasury>) -> Result<()> {
38     SetTreasury::apply(&mut ctx)
39 }
40
41 pub fn set_fees(mut ctx: Context<SetFees>, params: SetFeesParams) -> Result<()> {
42     SetFees::apply(&mut ctx, &params)
43 }
44
45 pub fn set_pause(mut ctx: Context<SetPause>, params: SetPauseParams) -> Result<()> {
46     SetPause::apply(&mut ctx, &params)
47 }
48
49 pub fn add_operator(mut ctx: Context<AddOperator>, _operator: Pubkey) -> Result<()> {
50     AddOperator::apply(&mut ctx)
51 }
```

Remediation To mitigate the identified issue, it is recommended to introduce multi-sig mechanism to undertake the role of the privileged account. Moreover, it is advisable to implement timelocks to govern all modifications to the privileged operations.

Response By Team This issue has been confirmed by the team.

3.3.4 [L-1] Revisited Chainlink Controller in set_price()

Target	Category	IMPACT	LIKELIHOOD	STATUS
set_price.rs	Business Logic	Low	Low	Addressed

The annotation suggests the constraint should use [CHAINLINK_ACCESS_CONTROLLER_MAINNET](#) rather than the devnet constant for mainnet deployments, risking misconfiguration or failed verification on the wrong cluster.

```
pandora-solana-contracts-main - set_price.rs
52 pub fn apply(ctx: &mut Context<SetPrice>, params: SetPriceParams) -> Result<()> {
53     let poll = &mut ctx.accounts.poll_15m;
54
55     let (expected_verifier_account, _) = Pubkey::find_program_address(
56         &[CHAINLINK_VERIFIER_ACCOUNT_SEED],
57         &ctx.accounts.chainlink_verifier_program.key(),
58     );
59     require!(
60         expected_verifier_account == ctx.accounts.verifier_account.key(),
61         ErrorCode::InvalidChainlinkVerifierAccount
62     );
63
64     require!(
65         ctx.accounts.access_controller.key() == CHAINLINK_ACCESS_CONTROLLER_DEVNET,
66         ErrorCode::InvalidChainlinkAccessController
67     );
68     ...
69
70     Ok(())
71 }
```

Remediation Derive the expected access controller from config or cluster, or document and enforce mainnet vs devnet explicitly in tests.

3.3.5 [L-2] Mandatory Opposite-Side ATA Blocks Single-Sided Sellers

Target	Category	IMPACT	LIKELIHOOD	STATUS
sell.rs	Business Logic	Low	Low	Addressed

The Sell accounts struct requires both [seller_yes_ata](#) and [seller_no_ata](#) to exist as initialized TokenAccounts (neither uses [init_if_needed](#)). At runtime the instruction only transfers from one of those ATAs per call, depending on [is_yes](#). If the user received YES or NO via a direct SPL transfer but never created the other mint's ATA, Anchor still tries to load both accounts at entry; deserialization fails and the user cannot sell at all.

```
pandora-solana-contracts-main - sell.rs

8
9 #[derive(Accounts)]
10 pub struct Sell<'info> {
11     ...
12     #[account(mut, associated_token::mint = yes_mint, associated_token::authority = seller)]
13     pub seller_yes_ata: Account<'info, TokenAccount>,
14
15     #[account(mut, associated_token::mint = no_mint, associated_token::authority = seller)]
16     pub seller_no_ata: Account<'info, TokenAccount>,
17
18     pub token_program: Program<'info, Token>,
19 }
20 impl Sell<'> {
21     pub fn apply(ctx: &mut Context<Sell>, params: &SellParams) -> Result<()> {
22         ...
23         let (seller_token_ata, target_vault) = if params.is_yes {
24             (
25                 ctx.accounts.seller_yes_ata.to_account_info(),
26                 ctx.accounts.yes_vault.to_account_info(),
27             )
28         } else {
29             (
30                 ctx.accounts.seller_no_ata.to_account_info(),
31                 ctx.accounts.no_vault.to_account_info(),
32             )
33         };
34         ...
35     }
36 }
```

Remediation Add [init_if_needed](#) to both seller ATAs (with [system_program](#) in the accounts struct), or only require the ATA that is actually used on the selected transaction path.

3.3.6 [I-1] Revisited Privileged Account Typing

Target	Category	IMPACT	LIKELIHOOD	STATUS
set_config.rs	Business Logic	Low	Low	Addressed

In the current implementation, `new_owner` (`ChangeOwner`) and `treasury` (`SetTreasury`) are constrained as `SystemAccount<'info>` with `lamports() > 0`, while the authorizing owner is a `Signer` (typically a single key). This design restricts privileged destination addresses to system-owned accounts only and explicitly excludes program-owned or PDA-controlled governance addresses by type. As a result, the current implementation is not compatible with production setups that require on-chain multisig (e.g. Squads) or programmatic custody for the owner, treasury, or `new_owner` roles.

```
pandora-solana-contracts-main - set_config.rs

52 #[derive(Accounts)]
53 pub struct ChangeOwner<'info> {
54     #[account()]
55     pub owner: Signer<'info>,
56     #[account(
57         mut,
58         has_one = owner @ ErrorCode::Unauthorized,
59     )]
60     pub config: Account<'info, Config>,
61     #[account(
62         constraint = new_owner.lamports() > 0
63     )]
64     pub new_owner: SystemAccount<'info>,
65 }
66 impl ChangeOwner<'_> {
67     pub fn apply(ctx: &mut Context<ChangeOwner>) -> Result<()> {
68         ctx.accounts.config.owner = ctx.accounts.new_owner.key();
69         Ok(())
70     }
71 }
```

Remediation Use more flexible account constraints for `new_owner` and `treasury` (instead of `SystemAccount`-only) so PDA/program-owned multisig addresses are supported natively.

4 Appendix

4.1 About AstraSec

AstraSec is a premier blockchain security firm dedicated to delivering high-quality auditing services for blockchain-based protocols. Composed of veteran security researchers with extensive experience across the DeFi landscape, our team maintains an unwavering commitment to excellence and precision. AstraSec’s comprehensive methodology and deep understanding of blockchain architecture enable us to ensure protocol resilience, making us a trusted partner for our clients.

4.2 Disclaimer

The content of this audit report is for informational purposes only and does not constitute legal, financial, or investment advice. The findings, views, and conclusions presented herein are based on the code and documentation provided at the specific time of the audit and may be subject to risks and uncertainties not detected during the assessment.

While AstraSec exerts every effort to ensure the accuracy and completeness of this report, the information is provided on an "as is" basis. We assume no responsibility for any errors, omissions, or inaccuracies. Users should conduct their own independent due diligence and consult with professional advisors before making any investment or deployment decisions. AstraSec shall not be held liable for any direct or indirect losses, damages, or consequences arising from reliance on this report.

4.3 Contact

Phone	+86 156 0639 2692
Email	contact@astrasec.ai
X	https://x.com/AstraSecAI